



Performance & Scalability:

Inside the SAP & Intel Co-engineering Journey

Jan Krüger

Thomas Willhalm

SAP x Intel Co-Development Initiative for HANA Cloud Platform

- In June 2026, Intel x SAP announced a dedicated SAP HANA Cloud Co-Development Initiative expanding our long-standing collaboration.
- This is the start of a new chapter in how Intel and SAP co-innovate: aligning SAP's product direction with Intel's platform roadmap, so optimizations show up where they matter most, at the customers and developers.



More than a decade of co-innovation of SIMD-processing in SAP HANA

2009

- “SIMD-scan. Ultra Fast in-Memory Table Scan using on-Chip Vector Processing Units” [VLDB 2009]

2013

- Intel introduces AVX2 with Intel Xeon E5 v3 processors
- “Vectorizing Database Column Scans with Complex Predicates” leveraging AVX2 [ADMS 2013]

2017

- Intel launches Intel Xeon Scalable processors with Intel AVX-512 instructions

2021

- Vector bit-manipulation instructions (VBMI) with 3rd gen Intel Xeon processors (Code-named Ice Lake)



Agenda

- Improving SAP HANA NLP service with Intel Advanced Matrix Extensions
- Efficient memory access for Vector Search
- Improving scalability for transactional workloads with lock-free synchronization methods
- AVX-512 optimizations of “Sorted-Set Intersection”

Acknowledgements:

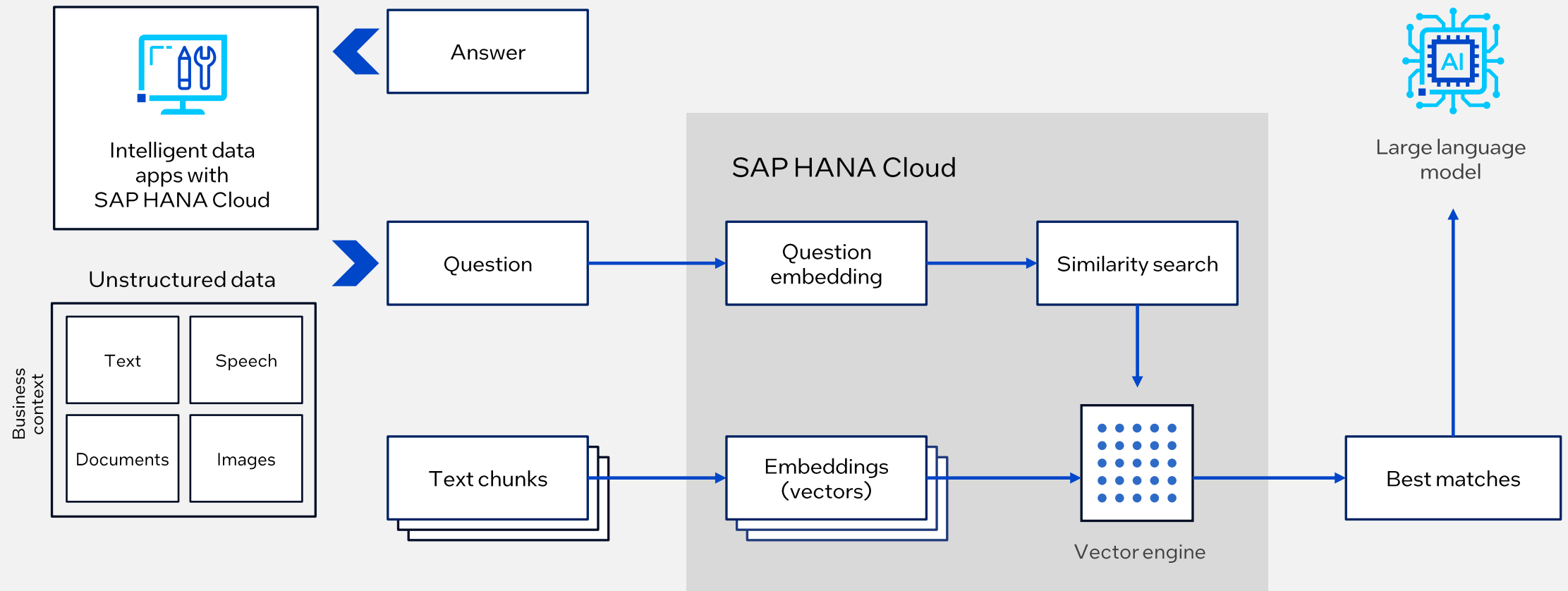
Lars Hoemke, Roman Dementiev, Christina Teflioudi, Alan Braenzel, Pavithran Pandiyan, Oliver Horn, Nicolae Popovici, Pawel Palucki, Brook Zhao, Justin Huang, Jiacan Xu

Improving SAP HANA NLP service **with Intel Advanced Matrix Extensions**

Taking advantage of built-in CPU accelerators

Retrieval augmented generation

With SAP HANA Cloud



Intel® Advanced Matrix Extensions (Intel® AMX) DL Accelerator Performance Built Into Every Core



Accelerating SAP NLP on Intel CPUs



SAP NLP runs productively on Intel® Xeon® CPUs, delivering proven performance at scale



Delivered strong CPU value for SAP NLP services with **~2× performance gains** using AMX and INT8, improving scalability and cloud TCO.



Shaping SAP's AI stack toward Intel-optimized frameworks by optimizing runtime, quantization, and threading best practices.



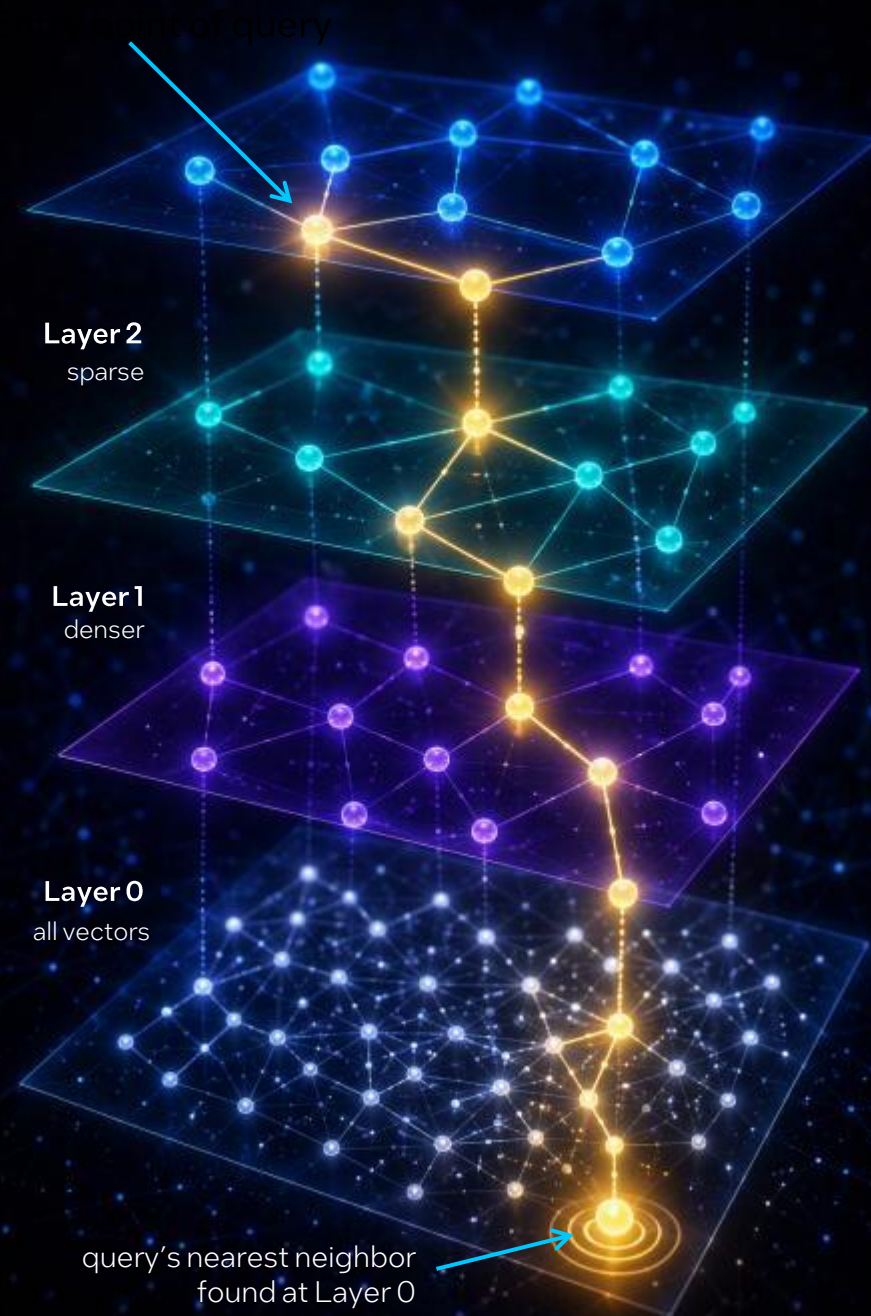
Efficient memory access for Vector Search

NUMA locality and prefetcher optimizations

Hierarchical Navigable Small World (HNSW)

A vector index built as a navigable small-world graph, stacked into layers for fast approximate nearest-neighbor search.

- HNSW (Hierarchical Navigable Small World) index construction builds navigable graphs.
- Each graph node holds a vector of floats (e.g. 768 dimensions → ~3 KB per vector)
- Traversing the graph computes the vector similarity between a node and all its neighbors (dozens to hundreds).
- Fixed source node (vector in L1 cache) vs. random neighbors (vectors fetched from DRAM).
- The hotspot is bound by stalls on vector loads.



HNSW index construction hotspot in a nutshell

Similarity Computation
is the top hotspot

Samples: 6M of event 'cycles:pp', Event count (approx.): 3989851526362

Overhead	Command	Shared Object	Symbol
68.59%	HanaWorker	libhdbvector.so	[.] CosineSimilarity
19.19%	HanaWorker	libhdbvector.so	[.]
5.22%	HanaWorker	libhdbvector.so	[.]
2.67%	HanaWorker	libhdbvector.so	[.]
0.90%	HanaWorker	libhdbvector.so	[.]
0.68%	HanaWorker	libhdbbasis.so	[.]

Cosine similarity (pseudocode)

```
for all neighbors v of node u:
    prefetch first cache lines of next neighbor
    cosine_similarity(u, v)
function cosine_similarity(u, v):
    for i from 1 to length:
        dot_product += u[i] * v[i]
        magnitude_u += u[i] * u[i]
        magnitude_v += v[i] * v[i]
    return dot_product / (sqrt(magnitude_u) * sqrt(magnitude_v))
```

*Pseudocode only — the real code processes data with AVX512
(three fused multiply-add operations per step).*

stall on vector
scan

three fused
multiply-add
operations

5.80	40:	nop
68.59		vmovups (%rdi,%rax,1),%zmm4
0.03		vmovups (%rsi,%rax,1),%zmm5
0.93		add \$0x40,%rax
1.88		vfmadd231ps %zmm4,%zmm4,%zmm0
0.45		vfmadd231ps %zmm5,%zmm5,%zmm2
0.00		vfmadd231ps %zmm5,%zmm4,%zmm1
0.01		sub \$0x10,%edx
0.02		jne 40
0.01	69:	vextractf64x4 \$0x1,%zmm0,%ymm3
0.02		vaddps %ymm3,%ymm0,%ymm0
0.02		vextractf128 \$0x1,%ymm0,%xmm3

NUMA awareness in HANA “index creation”

Index creation parallelized across all cores, not taking NUMA into account

Three steps make the index build NUMA-aware:

1

Partition the table in HANA to distribute its data across NUMA nodes

2

Add NUMA-node affinity to the jobs that compute similarity.

3

Mark those jobs as “no steal” so a neighbor node cannot take them.



Intel PCM: per-socket core stalls, memory bandwidth, request ratio, and UPI utilization across both sockets.

Result: index creation 2.54x faster by improved NUMA locality, mainly local memory requests, and memory bandwidth that reaches ~400 GB/s.

Using bandwidth efficiently – avoiding wasted memory prefetching

- **Data source are two vectors:**
 - a fixed source node (vector in L1 cache) and
 - random neighbors (vectors fetched from DRAM).
- Hardware prefetcher recognizes the stream access, and prefetches data from DRAM ahead of the loop.
 - but then keeps streaming past the end of the vector — a wasted prefetch.
- Memory bandwidth is the limiting resource, yet the four core hardware prefetchers waste up to 20%.
 - The waste is quantified with the L2_LINES_OUT.USELESS_HWPF event.
- Turning prefetchers “off” has negative impact on other code.

Forward scan (vector1, vector2)

```
function cosine_similarity(u, v):  
    for i from 1 to length:  
        dot_product += u[i] * v[i]  
        magnitude_u += u[i] * u[i]  
        magnitude_v += v[i] * v[i]
```

→ *prefetcher keeps streaming past length: wasted prefetch*

Processing of vector data (prefetched)

Unnecessarily
prefetched

VECTOR

PAST THE END

Taming the prefetcher: ends-to-middle (E2M)

- **Idea:** prefetch and process each vector from both ends toward the middle (E2M).
- The hardware prefetcher can recognize many streams — both backward and forward.
- The two streams meet in the middle and stop, because there is nothing left to prefetch.
- Rather than impairing the prefetcher's vision, E2M trains it to stop at the vector's fence.
- Useless prefetches drop from up to **20% to just 3–7% with E2M**.
- Performance matches or beats fully disabling the hardware prefetcher — **up to 20% speedup**.

Ends-to-middle (pseudocode)

```
function cosine_similarity(u, v):  
    for i from 1 to length/2:  
        dot_product += u[i] * v[i]  
        magnitude_u += u[i] * u[i]  
        magnitude_v += v[i] * v[i]  
        j = length + 1 - i  
        dot_product += u[j] * v[j]  
        magnitude_u += u[j] * u[j]  
        magnitude_v += v[j] * v[j]  
  
    // also process the middle element if length is odd
```

Processing of vector data (prefetched)

VECTOR

Improving scalability for transactional workloads with lock-free synchronization methods

Using Atomic Reads Instead of Compare-and-Swap

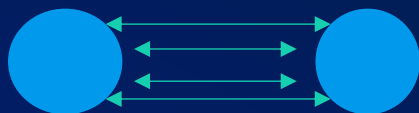
Scaling SAP HANA to 1000 cores and TB of memory

Official Name	Cores/HT per socket	OLAP capacity/ socket	OLTP capacity/ socket	max # of sockets	# of cores per system	HANA max memory capacity per system
2nd Gen Intel® Xeon® Scalable Processor (Cascade Lake)	28/56	768GB	1.5TB	16*	448	24TB
3rd Gen Intel® Xeon® Scalable Processor (Ice Lake)	40/80	1TB	2TB	2	80	4TB
4th Gen Intel® Xeon® Scalable Processor (Sapphire Rapids)	56/112	1.5TB	2TB	16*	896	32TB
5th Gen Intel® Xeon® Scalable Processor (Emerald Rapids)	64/128	2TB	3TB	2	128	6TB
Intel® Xeon® 6900P-series (Granite Rapids-AP)	128/256	3TB	3TB	2	256	6TB
Intel® Xeon® 6700P/6500P-series (Granite Rapids-SP)	86/172	4TB	4TB	16*	1376	64TB**

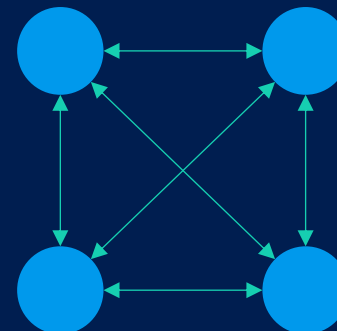
** HANA Cloud currently supports up to 24TB and HANA 2.0 up to 48TB for OLTP scale-up

Scaling servers from low end to high end

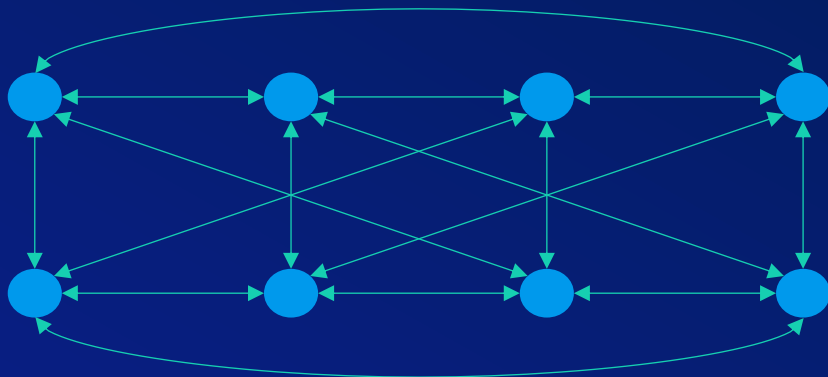
2 Intel® Xeon® processors



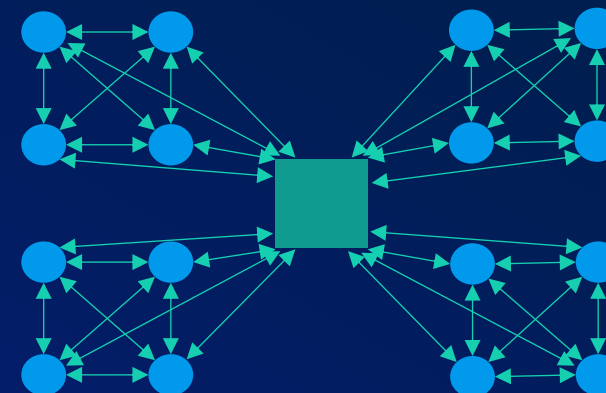
4 Intel® Xeon® processors



8 Intel® Xeon® processors



16 Intel® Xeon® processors with 3rd party node-controller



Compare-and-Swap vs. Atomic Reads

Compare-and-Swap (CAS) Functionality

CAS atomically checks and updates memory only if the expected value matches, ensuring correctness in concurrency.

Performance Costs of CAS

CAS requires exclusive ownership of cache lines, causing costly coherence traffic under contention.

Advantages of Atomic Reads

Atomic reads avoid cache invalidations and ownership transitions, leading to better scaling and lower overhead.

When to Use Atomic Reads

In scenarios requiring only state observation, atomic reads replace CAS for improved performance and scalability.

Cacheline exclusive in one cache



Cacheline shared in several caches



Replacing CAS with Atomic Loads



Optimization by Atomic Loads

Replacing CAS with atomic loads reduces overhead when no shared data modification is needed.



Scalability Benefits

Atomic loads enable multiple threads to read data concurrently, improving scalability in multi-core systems.



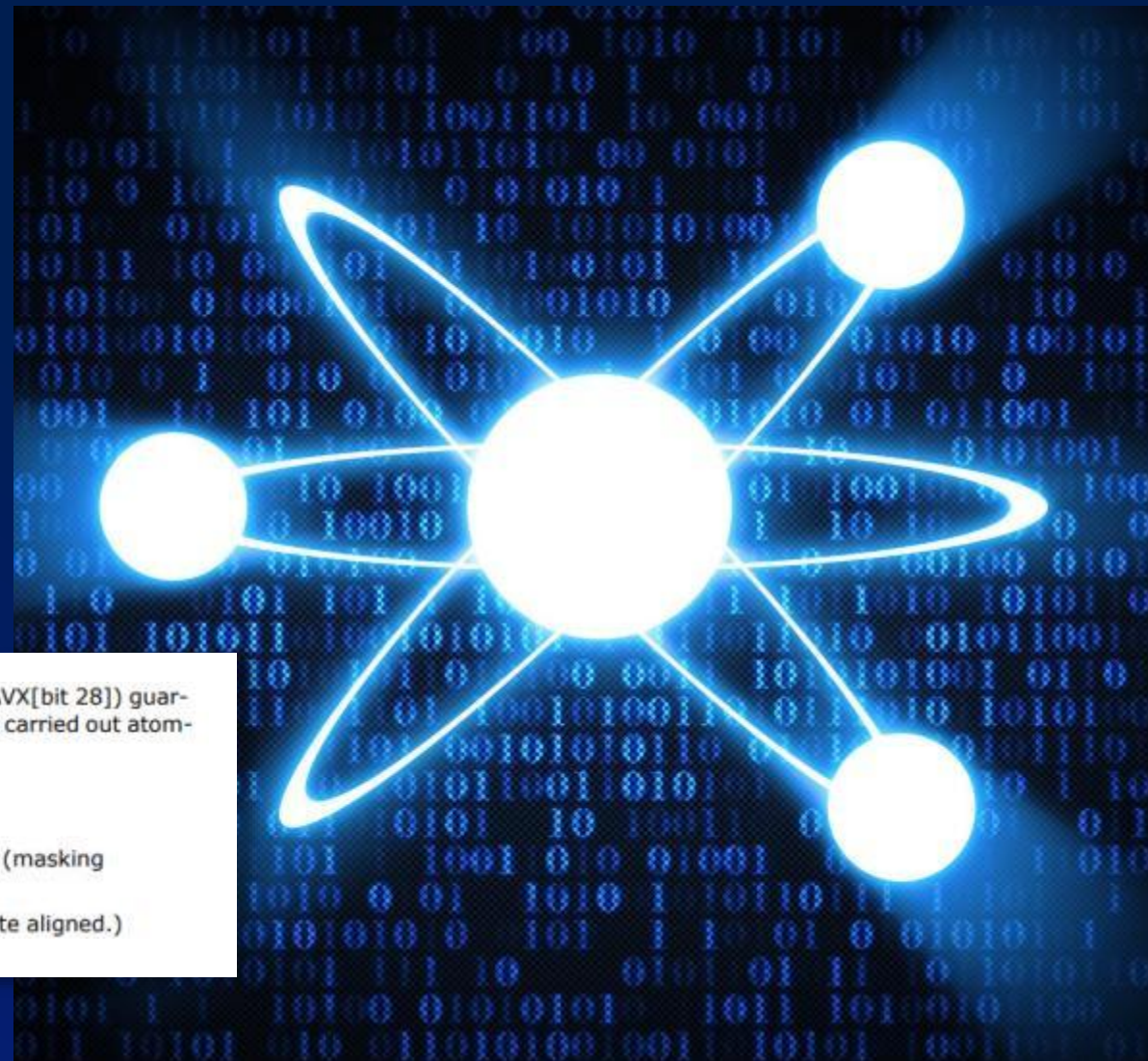
Performance and Power Efficiency

Using atomic loads simplifies code and reduces latency, throughput bottlenecks, and power consumption.

Processors that enumerate support for Intel® AVX (by setting the feature flag CPUID.01H:ECX.AVX[bit 28]) guarantee that the 16-byte memory operations performed by the following instructions will always be carried out atomically:

- MOVAPD, MOVAPS, and MOVDQA.
- VMOVAPD, VMOVAPS, and VMOVDQA when encoded with VEX.128.
- VMOVAPD, VMOVAPS, VMOVDQA32, and VMOVDQA64 when encoded with EVEX.128 and k0 (masking disabled).

(Note that these instructions require the linear addresses of their memory operands to be 16-byte aligned.)



Source: Intel 64 and IA-32 Architectures Software Developer's Manual

Intel VTune analysis of CAS and Atomic Reads

Hotspot “function_READ”

- 128bit header with atomic load used during every read
- Compiler uses use cmpxchg16b instruction for 128 bit atomic_load
- Large overhead with contention
- Significant performance impact when more cores are used.

Replacing CMPXCHG with READ

- Hot spot reduces from 12.8% to 7.3%
- Perf improves by 7.8% with 288 cores

Function / Call Stack	CPU Time	Clockticks	Instructions Retired	CPI Rate	Retiring
function_READ	12.80%	4,064,644,500,000	87,339,000,000	46.539	3.30%
function_AXSE	10.50%	3,324,436,500,000	30,271,500,000	109.82	1.20%
function_IVLX	7.10%	2,239,020,000,000	106,963,500,000	20.933	3.70%
function_THQB	5.80%	1,849,480,500,000	29,578,500,000	62.528	1.30%
function_CIZR	4.80%	1,477,255,500,000	1,234,600,500,000	1.197	26.10%
function_LZWX	3.80%	1,199,856,000,000	99,865,500,000	12.015	2.40%
function_BSLJ	2.50%	790,944,000,000	6,688,500,000	118.25	6.00%
function_XXXE	2.50%	781,924,500,000	323,452,500,000	2.417	13.10%
function_CDQW	1.70%	541,390,500,000	1,362,312,000,000	0.397	49.00%
function_CHFX	1.60%	496,660,500,000	51,429,000,000	9.657	1.80%
function_FRZV	1.50%	460,404,000,000	458,052,000,000	1.005	52.60%
function_MLTO	1.20%	394,411,500,000	93,481,500,000	4.219	5.10%
function_LZWX	1.20%	375,249,000,000	48,940,500,000	7.667	2.00%
function_TWPJ	1.00%	316,018,500,000	9,324,000,000	33.893	2.50%
function_TYHT	1.00%	313,855,500,000	30,838,500,000	10.177	7.30%
function_RKEQ	1.00%	314,296,500,000	197,589,000,000	1.591	15.00%
function_XBOC	0.80%	255,979,500,000	480,847,500,000	0.532	27.50%
function_KFHI	0.80%	257,187,000,000	28,360,500,000	9.068	5.70%

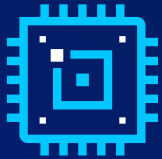
Function / Call Stack	CPU Time	Clockticks	Instructions Retired	CPI Rate	Retiring
function_AXSE	11.00%	3,066,367,500,000	30,481,500,000	100.6	1.50%
function_THQB	7.70%	2,165,068,500,000	28,969,500,000	74.736	2.40%
function_READ	7.30%	2,048,550,000,000	74,403,000,000	27.533	3.40%
function_IVLX	7.00%	1,946,742,000,000	105,451,500,000	18.461	4.40%
function_CIZR	5.20%	1,457,935,500,000	1,220,446,500,000	1.195	27.90%
function_LZWX	3.50%	982,180,500,000	92,263,500,000	10.645	2.80%
function_XXXE	3.50%	994,476,000,000	322,906,500,000	3.08	10.80%
function_BSLJ	2.70%	777,640,500,000	7,024,500,000	110.7	6.60%
function_CDQW	1.80%	490,780,500,000	1,299,028,500,000	0.378	51.70%
function_CHFX	1.70%	482,422,500,000	50,431,500,000	9.566	2.20%

Scaling transactional workloads to 1k cores



S4/HANA workload (pure OLTP)

Improvements in transaction management with atomic reads:



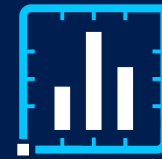
20%

lower CPU utilization



20%

lower response time
for the same number of users



10%

higher throughput

AVX-512 optimizations of “Sorted-Set Intersection”

When single functions dominate whole queries

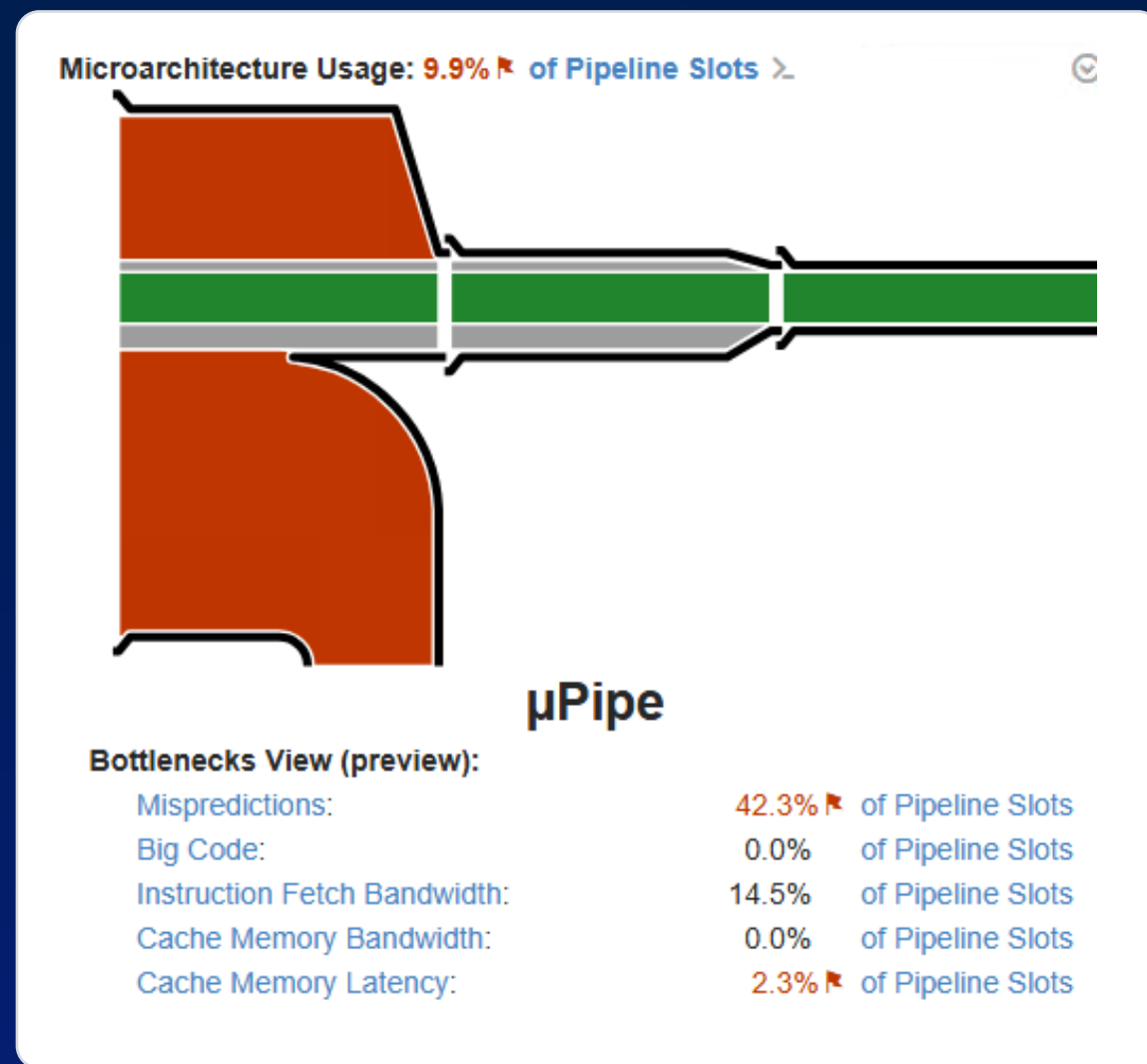
“Sorted-set intersection” hotspot

Found significant hotspots during queries in performance regression suite (up to 33%)

- Intersect_inplace_samerange routine
- Intersects two vectors after computation

VTune analysis on these queries showed

- Significant “bad speculation”, i.e. mispredicted branches
- Inherent to implementation as conditional jumps are data dependent



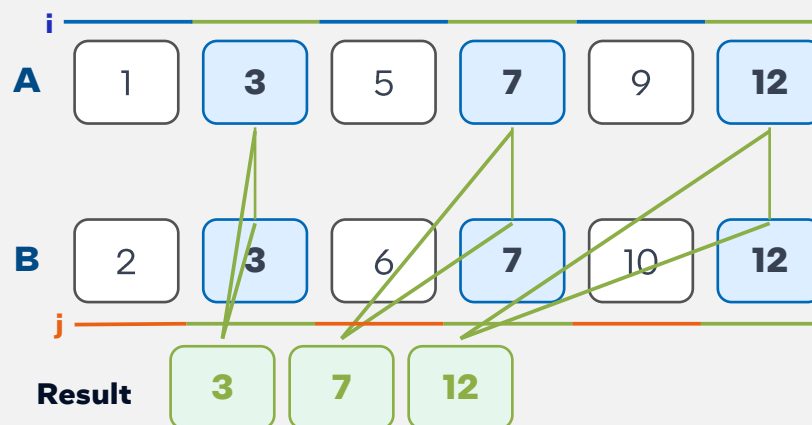
Sorted-Set Intersection: Two-Pointer Merge Scan

Pseudo-code implementation

```
intersect(A, B):  
    i, j = 0, 0  
    out = []  
    while i < len(A) and j < len(B):  
        if A[i] == B[j]:  
            out.append(A[i])  
            i += 1; j += 1  
        elif A[i] < B[j]:  
            i += 1  
        else:  
            j += 1  
    return out
```

Invariant: because both inputs are sorted, the smaller current value can never match later values in the other set — advance only that pointer.

Visualization with detailed pointer movements



Pointer movement sequence



— advance A pointer

— advance B pointer

— equal → emit + advance both

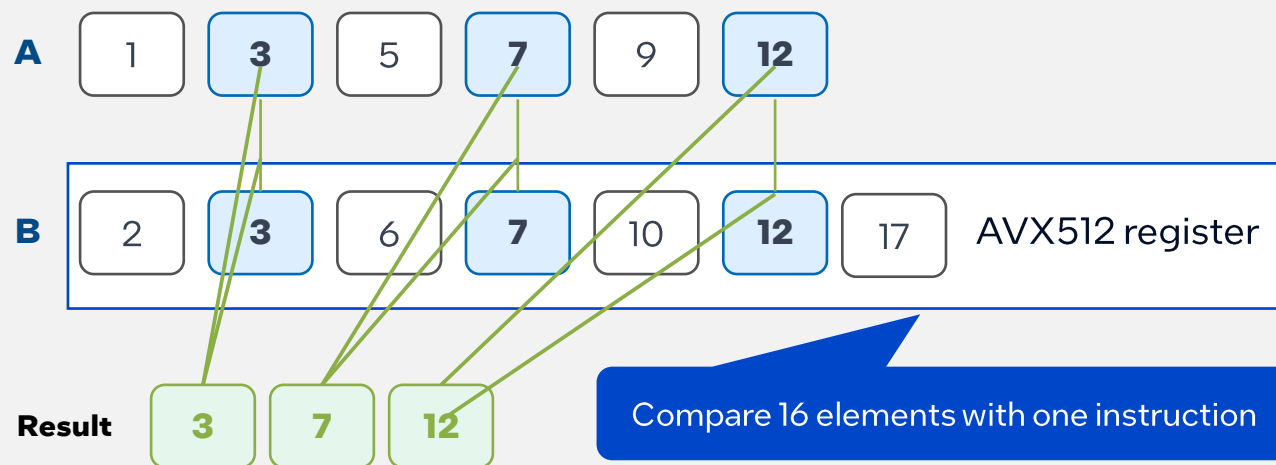
Sequential access pattern is cache-friendly, but branches are data-dependent and unpredictable.

“Sorted-set intersection” with Intel AVX-512 instruction

TWO APPROACHES:

A. Sequential SIMD Compare:

Taking every element in first array in the AVX-512 register and compare in-parallel against the second array



B. Implementation full 16x16 comparison with AVX-512

“vp2intersect” instruction with a software sequence

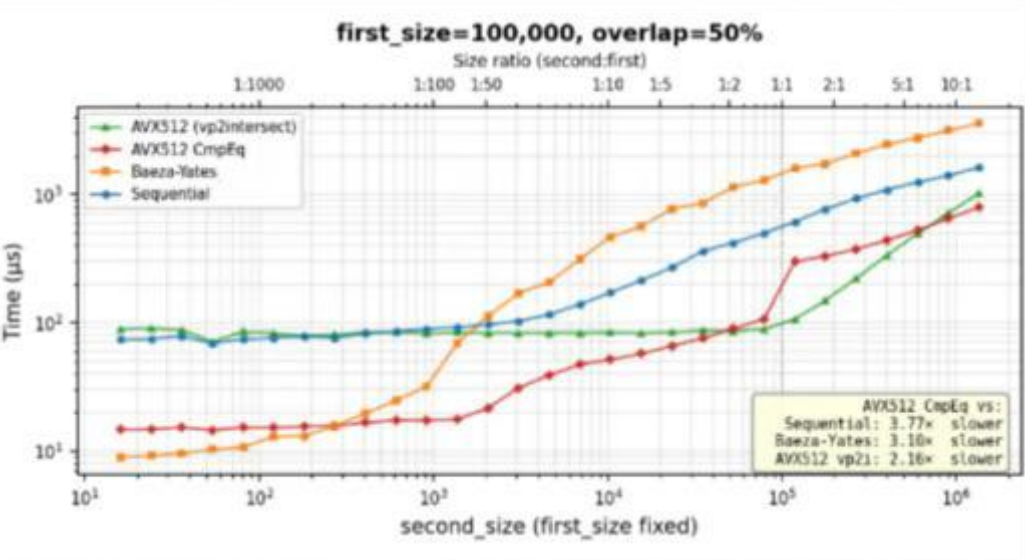
Reference: [“Faster-Than-Native Alternatives for x86 VP2INTERSECT Instructions”](#)

1. First, shift input by lane and within lanes
2. Then, 16 comparisons with these two registers implements an exhaustive comparison between all elements



Impact of AVX512 Compare

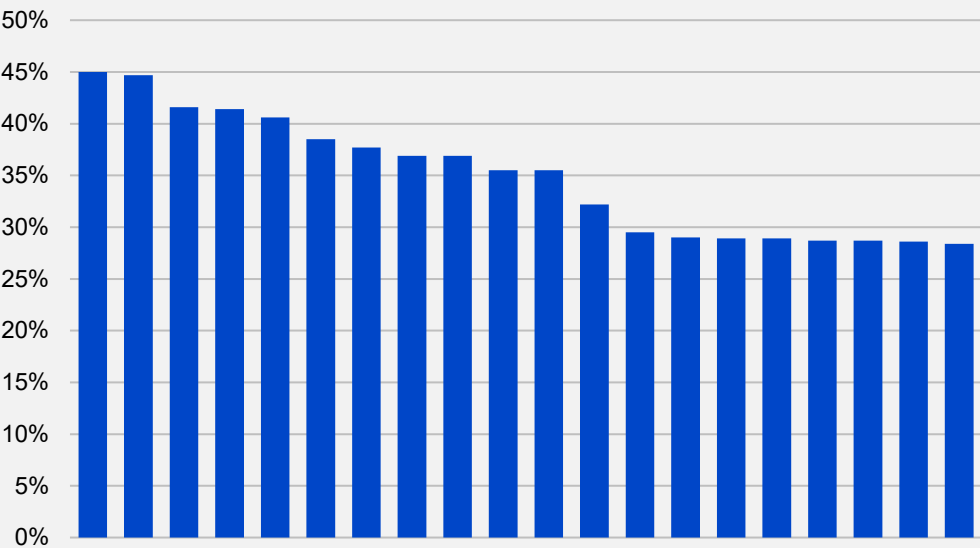
Sequential SIMD Compare



- Faster than vp2intersect emulation for array size 200k
- Faster than Baeza-Yates Intersection in the same range for smaller arrays.

Depending on input sizes and overlap, one implementation outperforms others

Top 20 Query Improvements



End-to-end, queries using sorted-set intersection improved by up to 45%!

Key Takeaways

1

Use the accelerators already in the core. Intel AMX with INT8 doubled SAP HANA NLP throughput — **no GPU required.**

2

Locality is an optimization. Making the vector-index build NUMA-aware delivered a **2.5× speedup.**

3

Train the prefetcher, don't fight it. Ends-to-middle scanning cut wasted bandwidth from **20% to under 7%.**

4

Don't pay for a write you never make. Atomic reads instead of CAS cut CPU use and response time **20% at 1,000+ cores.**

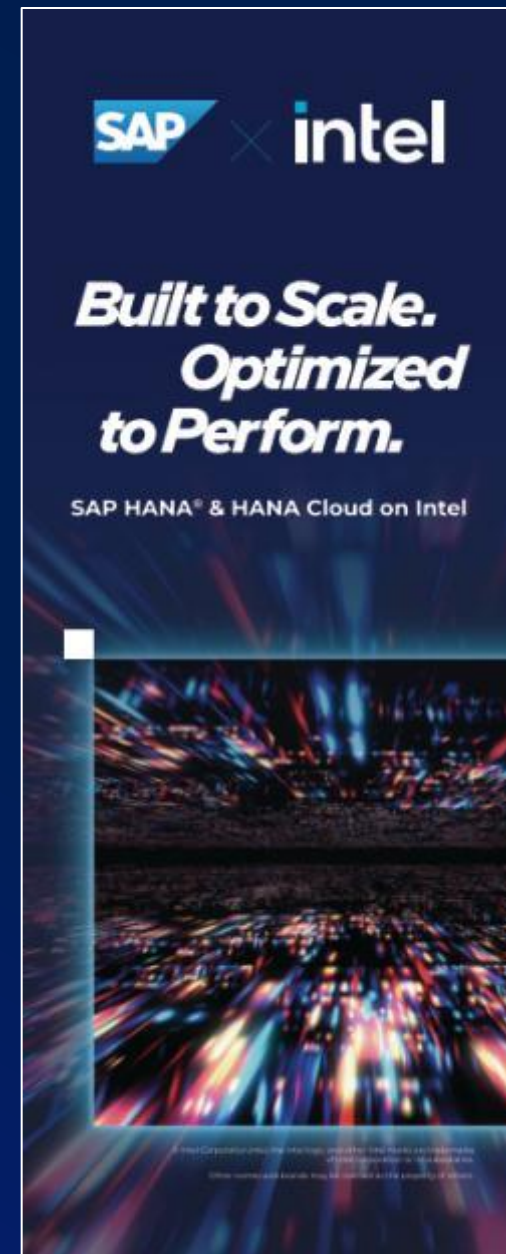
5

One hot function can move the whole query. AVX-512 sorted-set intersection sped up queries by **up to 45%.**

Thank you.

And meet us at the booth.

- Continue the Discussion
- Further insights into benefits to you
- Get your Intel x SAP merchandise



The Intel logo is centered on a dark blue background. It features the word "intel" in a white, lowercase, sans-serif font. A small, solid blue square is positioned above the first vertical stroke of the letter 'i'. To the right of the word "intel" is a small registered trademark symbol (®).

intel®